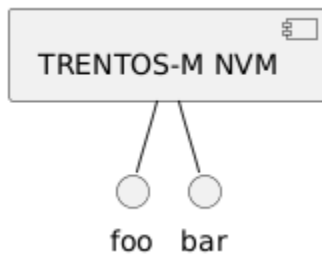


TRENTOS-M NVM

- [Overview](#)
- [Library functions and structures](#)
 - [Raw C API](#)
 - [Object oriented API](#)
 - [TRENTOS-M NVM CAmkES component API](#)
- [Implementations](#)
 - [ProxyNVM](#)
 - [Communication Protocol](#)
 - [ToDo](#)
 - [Return Codes](#)
 - [Commands](#)
 - [Context](#)
 - [Functions](#)
 - [ProxyNVM_write](#)
 - [ProxyNVM_eraseProxyNVM_read](#)
 - [ProxyNVM_erase](#)
 - [ProxyNVM_getSize](#)
 - [AesNVM](#)
 - [Configuration](#)
 - [Context](#)
 - [internal functions](#)
 - [public API](#)
 - [AesNvm_write](#)
 - [AesNvm_read](#)
 - [AesNvm_erase](#)
 - [AesNvm_getSize](#)

Overview

- the library implements an abstract interface that defines methods and structures for interacting with lower layer drivers handling non-volatile memory
- **ToDo:** describe concept of the library, use cases, architecture diagram, how the library is used in a system ...



Library functions and structures

Raw C API

ToDo: describe API

Object oriented API

NVM class

```
typedef struct Nvm Nvm;

typedef size_t
(*Nvm_WriteT)(Nvm* self, size_t addr, char const* buffer, size_t length);

typedef size_t
(*Nvm_ReadT)(Nvm* self, size_t addr, char* buffer, size_t length);

typedef size_t
(*Nvm_EraseT)(Nvm* self, size_t addr, size_t length);

typedef size_t
(*Nvm_GetSizeT)(Nvm* self);

typedef void
(*Nvm_DtorT)(Nvm* self);

typedef struct
{
    Nvm_WriteT    write;
    Nvm_ReadT     read;
    Nvm_EraseT    erase;
    Nvm_GetSizeT  getSize;
    Nvm_DtorT     dtor;
}
Nvm_Vtable;

struct Nvm
{
    const Nvm_Vtable* vtable;
};
```

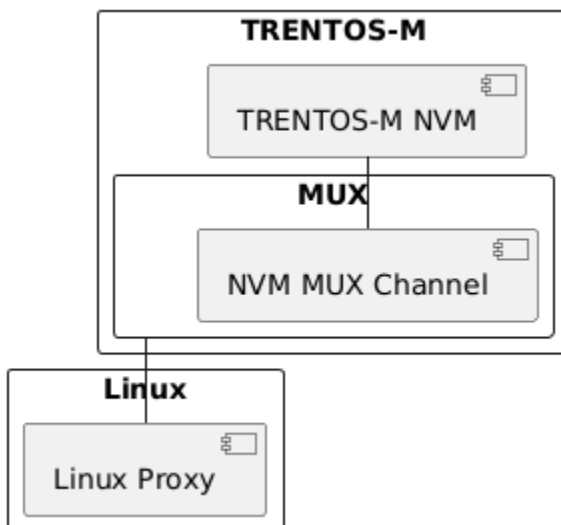
TRENTOS-M NVM CAMkES component API

ToDo: describe API

Implementations

ProxyNVM

- An implementation of the abstract NVM interface that is using the ChanMuxClient as a lower level interface



- Communicates with the mqtt_proxy_demo via **channel 6** which uses a file as a non-volatile memory
- The size of the created file is hard-coded in the mqtt_proxy_demo and cannot be changed from the ProxyNVM side

Communication Protocol

- The protocol uses a synchronous request-response scheme.
- It uses raw data and assumes there is an underlying transport protocol, that ensures data integrity
- there is no partition ID in the protocol, multiple communication channels must be set up, each related to exactly one partition

ToDo

- improve protocol
 - add erase request
 - could use sequence numbers for each command, so each response can be connected to exactly one request, even if the same command is sent multiple times in a row
 - generic command format that allows adding new commands without breaking things

```
Request: [Sequence number] [2 byte length payload as big endian integer] [ Payload: [1 byte
Command=0] [...data...] ]
Response: [Sequence number] [2 byte length payload as big endian integer] [ Payload: [1 byte
Retval] [...data...] ]
```

Return Codes

Name	Value (1 byte)	Description
OK	0x00	operation successful
GENERIC_ERROR	0xFF	operation failed
INVALID_COMMAND		

Commands

Command	Value (1 byte)	Description
get_size	0x00	get size of partition
write	0x01	write data buffer to address in partition
read	0x02	read data buffer from address in partition
erease	0x03	erase a block (partition memory type implementation specific)

Protocol

-----GetSize-----

Request: [1 byte Command=0]

Response: [1 byte Command=0] [1 byte Retval] [4 byte size in big endian integer if RetVal=0]

Example: Get the capacity of the NVM

Request: 0x00

Response: 0x00 0x00 0x00 0x00 0x00 0x80

-----Write-----

Request: [Command=1] [4 byte address as big endian integer] [4 byte length as big endian integer] [... data...]

Response: [Command=1] [Retval] [4 byte length written as big endian integer]

Example: Write 2 bytes 0xAA and 0x55 from address 0x00000080

Request: 0x01 0x00 0x00 0x00 0x80 0x00 0x00 0x00 0x02 0xAA 0x55

Response: 0x01 0x00 0x00 0x00 0x00 0x02

-----Read-----

Request: [Command=2] [4 byte address as big endian integer] [4 byte length to read as big endian integer]

Response: [Command=2] [Retval] [4 byte length read as big endian integer] [...data...]

Example: Read 2 bytes from address 0x00000080

Request: 0x01 0x00 0x00 0x00 0x80 0x00 0x00 0x00 0x02

Response: 0x02 0x00 0x00 0x00 0x00 0x02 0xAA 0x55

-----Erase-----

Request: [Command=3] [4 byte address as big endian integer] [4 byte length to read as big endian integer]

Response: [Command=3] [Retval]

Context

ProxyNVM

```
struct ProxyNVM
{
    Nvm parent;
    ChanMuxClient* chanmux;
    size_t size;
    char* msgBuf;
    size_t msgBufSize;
};
```

Field	Explanation
parent	Parent context (vtable)
chanmux	Pointer to the ChanMuxClient object which is used for communication with mqtt_proxy_demo
size	Size of the memory which is initialized in the constructor by executing the GetSize request of the protocol
msgBuf	Pointer to the memory block that is to be used as an internal message buffer (if the ChanMuxDataport is used, unnecessary memcpy calls can be avoided)
msgBufSize	Size of the msgBuf

Functions

ProxyNVM_write

ProxyNVM_write

```
size_t
ProxyNVM_write(Nvm* nvm,
               size_t addr,
               char const* buffer,
               size_t length)
```

- Functional description
 - Creates the correct header for the write request (protocol) and puts it to the message buffer
 - Copies the max amount of bytes (determined by the msgBufSize) from buffer to the msgBuf (or length amount if length < max length)
 - Executes the write command
 - Repeat until the entire buffer is written
- Return values
 - Returns the number of successfully written bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not there has been an error)

ProxyNVM_eraseProxyNVM_read

ProxyNVM_read

```
size_t
ProxyNVM_read(Nvm* nvm,
              size_t addr,
              char* buffer,
              size_t length)
```

- Functional description
 - Creates the correct header for the read request (protocol) and puts it to the message buffer
 - Executes the read command
 - Copies the max amount of bytes (determined by the msgBufSize) from the msgBuf to the buffer (or length amount if length < max length)
 - Repeat until all of the bytes are read
- Return values
 - Returns the number of successfully read bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not there has been an error)

ProxyNVM_erase

ProxyNVM_erase

```
size_t
ProxyNVM_erase(Nvm* nvm,
               size_t addr,
               size_t length)
```

- Functional description
 - Behaves the same way as the ProxyNVM_write, with the difference that it writes 0xFF to the block at the desired address with the desired size
- Return values
 - Returns the number of successfully erased bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not there has been an error)

ProxyNVM_getSize

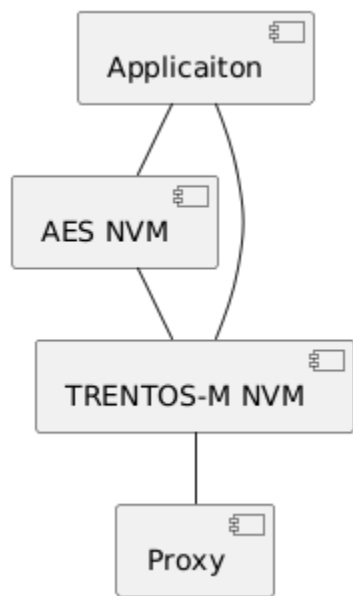
ProxyNVM_getSize

```
size_t
ProxyNVM_getSize(Nvm* nvm)
```

- Functional description
 - Executes the GetSize request (protocol)
- Return values
 - Returns the size of the memory in bytes

AesNVM

- An implementation of the abstract NVM interface that is using another NVM implementation as the lower level interface



- Implements AES block encryption on top of the lower level NVM interface using the TRENTOS-M Crypto interface
- Uses the [ESSIV](#) method for block encryption, individual keys that encrypt each block are derived from a master key using the block base address
- All functions **allocate work buffer on the stack**, where the pages are processed that are affected of the current implementation

Configuration

Hardcoded configuration of the AesNVM			
#define KEY_LENGTH	32	// AES-256	
#define IV_LENGTH	16	// AES uses 16 byte blocks	
#define ENCRYPTION_PAGE_LEN	1024	// 1 KiB is chosen as good trade off for a file system with small files	

ToDo:

- master key is currently hard coded, should be configurable to use a TRENTOS-M Keymanager Key ID.

Context

AesNVM structure
<pre>struct AesNvm { Nvm parent; Nvm* underlyingNvm; };</pre>

internal functions

All of the functions internally call `cryptoCalculateBlock()`, which calculates the IV for each block based on the address of the block using the ESSIV method and encrypts/decrypts the block depending on the operation parameter.

cryptoCalculateBlock

```
static void
cryptoCalculateBlock(
    size_t addr,
    char* input,
    char* output,
    uint8_t operation)
```

public API

AesNvm_write

AesNvm_write

```
size_t
AesNvm_write(
    Nvm* nvm,
    size_t addr,
    char const* buffer,
    size_t size)
```

- Functional description
 - Calculates the beginning of the encryption block for the given address
 - Reads the first encryption block
 - Decrypts the block
 - Writes the plain text from the buffer to the decrypted block
 - Encrypts the changed block
 - Writes the encrypted block to memory by calling the lower level nvm_write command
 - If there is more to be written, moves the address to the next encryption block and repeats the algorithm
- Return values
 - Returns the number of successfully written bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not, there has been an error)

AesNvm_read

AesNvm_read

```
size_t
AesNvm_read(
    Nvm* nvm,
    size_t addr,
    char* buffer,
    size_t size)
```

- Functional description
 - Calculates the beginning of the encryption block for the given address
 - Reads the first encryption block
 - Decrypts the block
 - Copies the requested part of the current encryption block to the buffer
 - If there is more to be read, moves the address to the next encryption block and repeats the algorithm
- Return values
 - Returns the number of successfully read bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not, there has been an error)

AesNvm_erase

AesNvm_erase

```
size_t
AesNvm_erase(
    Nvm* nvm,
    size_t addr,
    size_t size)
```

- Functional description
 - **Min erase size is ENCRYPTION_PAGE_LEN**
 - Calculates the beginning of the encryption block for the given address
 - Encrypts the block populated with 0xFF values
 - Writes the encrypted block to memory by calling the lower level nvm_write command
 - If there is more to be erased, moves the address to the next encryption block and repeats the algorithm
- Return values
 - Returns the number of successfully erased bytes
 - The caller should check whether the returned number of bytes matches the requested length (if not, there has been an error)

AesNvm_getSize

AesNvm_getSize

```
size_t
AesNvm_getSize(
    Nvm* nvm)
```

- Functional description
 - Executes the GetSize command of the lower level nvm_getSize
- Return values
 - Returns the size of the memory in bytes